

Database Filters, Indexes, and Hashing

Filters vs. Indexes

Databases use both filters and indexes to avoid scanning entire tables when searching for data, but they serve different purposes.

An **index** tells the database *where* the data is located. It acts like a map that connects a key to the exact records that contain that key. Common index structures include B+ Trees and Hash Tables.

A **filter**, on the other hand, only answers the question: "*Does this key probably exist?*" It does not store the location of the data. Filters are often probabilistic, meaning they may occasionally say a key exists when it does not (false positive), but they will never say a key does not exist when it actually does (no false negatives).

Bloom Filters

A Bloom Filter is one of the most widely used filters.

It consists of:

- A fixed-size bitmap (an array of bits)
- Multiple hash functions

Inserting a Key

When a key is inserted, it is passed through several hash functions. Each hash function produces a position in the bitmap, and those positions are set to 1.

For example:

Key = "Apple"

Hash Function 1 → Bit 3

Hash Function 2 → Bit 8

Hash Function 3 → Bit 15

The filter sets bits 3, 8, and 15 to 1.

Searching for a Key

The same hash functions are applied to the search key.

- If any corresponding bit is 0, the key does not exist.

- If all corresponding bits are 1, the key might exist.

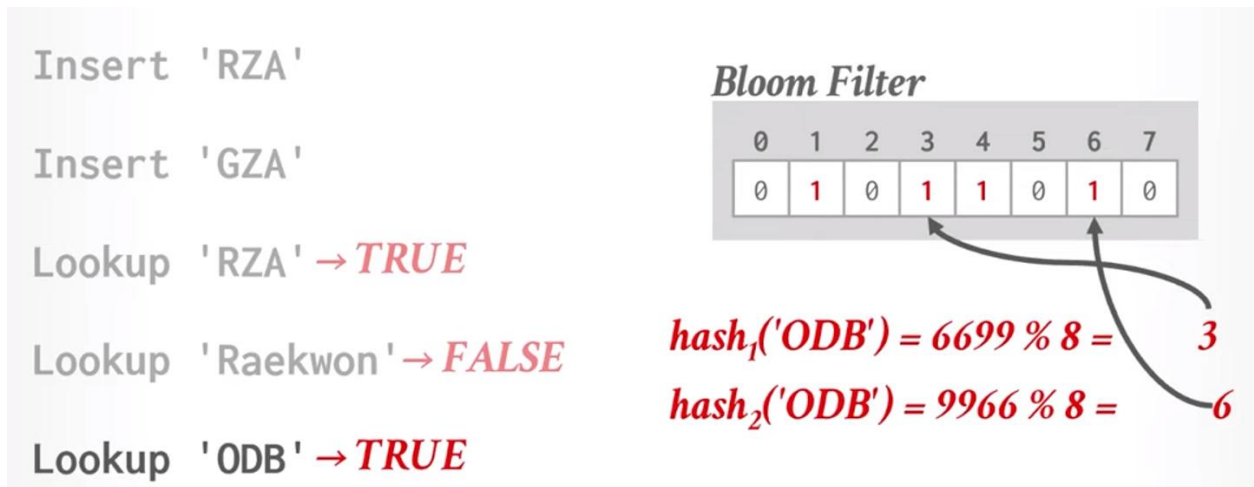
This uncertainty leads to false positives.

False Positives

Different keys can set some of the same bits. As more keys are inserted, the bitmap becomes crowded, increasing the chance that an unrelated key appears to exist.

The false-positive rate depends on:

- Bitmap size
- Number of hash functions
- Number of inserted keys



Skip Lists

A Skip List is a probabilistic data structure that behaves like an in-memory index.

At the bottom level is a normal sorted linked list containing every key.

Above this are additional levels containing "skip pointers" that allow searches to jump over large sections of data.

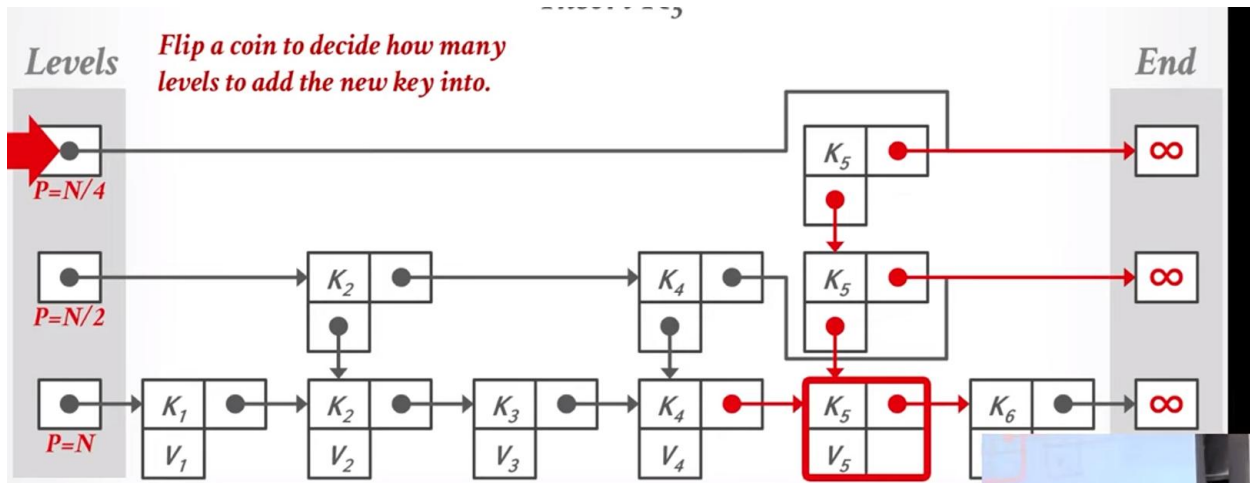
How It Works

Instead of performing expensive balancing operations like a B+ Tree, Skip Lists use randomness.

When a new key is inserted:

1. The key is added to the bottom level.
2. A coin is flipped repeatedly.
3. Each successful flip creates another level in the tower.

A few keys end up with tall towers, while most have short towers.



Performance

Searching typically takes:

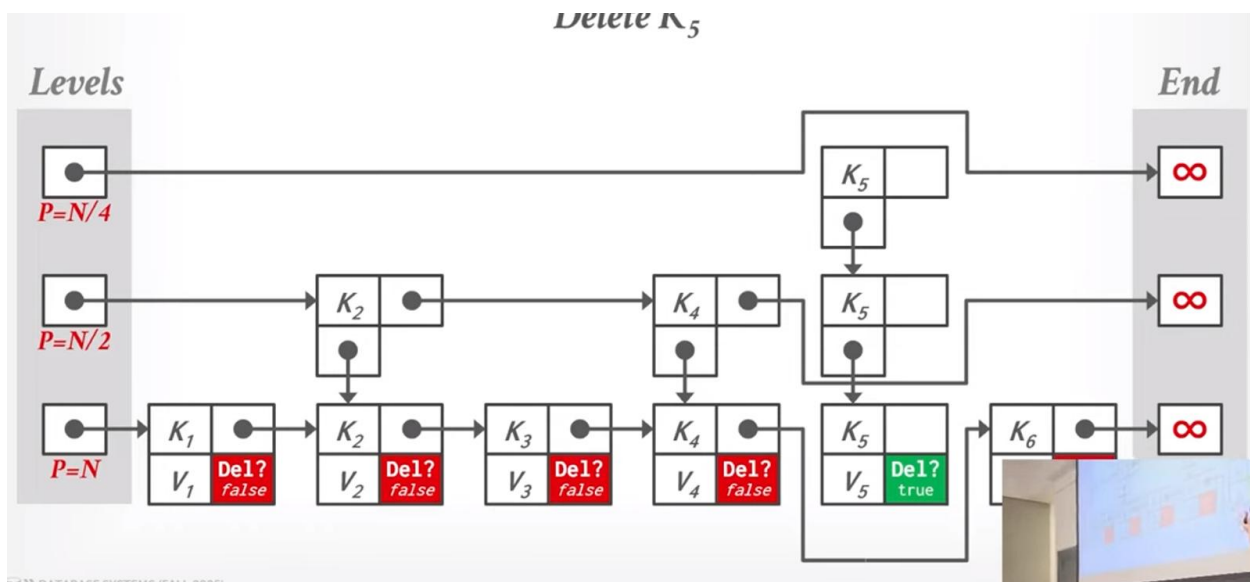
$O(\log N)$

which is comparable to balanced tree structures.

Deletions

Skip Lists often use tombstones (logical deletion markers). A background cleanup process later removes the actual nodes.

In a Skip List, a **delete bit (del bit)** or **tombstone** is used to mark a node as deleted without immediately removing it from all levels of the structure.



Why keep a del bit in skip lists? OR Why not delete the node immediately?

A key can appear in multiple levels:

Level 3: 20 -----> 50

|

Level 2: 20 ---> 35 -> 50

|

Level 1: 20 -> 25 -> 35 -> 50

If you delete 35, you'd have to:

1. Find all copies of 35 in every level.
2. Update all neighboring pointers.
3. Handle concurrency issues if other threads are currently traversing the list.

This can be expensive and complicated.

Common Use Cases

Skip Lists are mainly used for in-memory systems such as MemTables in storage engines because disk-based implementations would generate excessive I/O.

Tries and Radix Trees

A Trie is short for "retrieval tree."

Instead of storing complete keys in each node, a Trie stores one character, digit, or bit per level.

For example, the words:

Cat

Car

would share the path:

C → A

before branching into:

T

R

Advantages

- Fast lookups
- Early termination when a path does not exist
- No balancing operations required

The structure is determined entirely by the keys being stored.

Radix Trees

A Radix Tree is a compressed Trie designed to reduce memory usage.

It performs two types of compression:

Horizontal Compression

Removes unnecessary empty child pointers.

Vertical Compression

Collapses long chains of single-child nodes into a single jump.

For example:

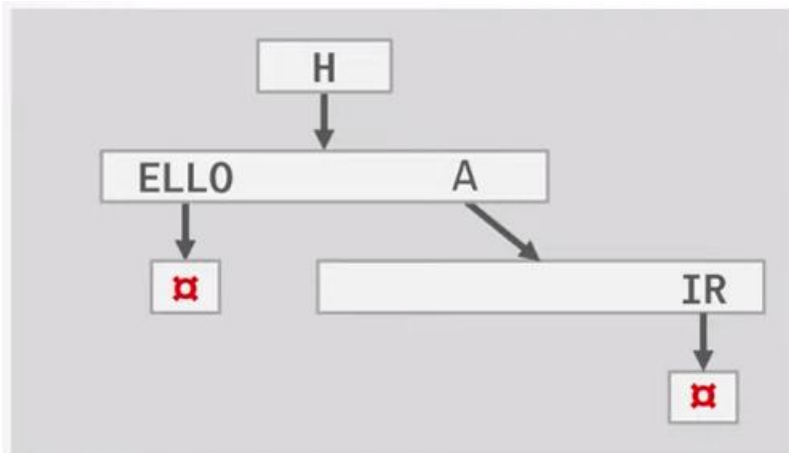
C → O → M → P → U → T → E → R

can become:

COMPUTER

stored in a single compressed edge.

This significantly reduces memory consumption and improves traversal speed.



Inverted Indexes (Full-Text Search)

Traditional B+ Tree indexes work well when searching entire column values.

However, they struggle with text searches such as:

Find every document containing the word "database".

Scanning every row would be slow.

Inverted Index

An inverted index breaks text into individual words (terms) and creates a dictionary.

Each term points to a posting list containing the IDs of records where the term appears.

Example:

Database → [5, 8, 20]

means records 5, 8, and 20 contain the word "database".

Popular Implementations

Lucene

Use a Finite State Transducer (FST), a highly compressed trie-like structure optimized for text search.

PostgreSQL GIN Indexes

Use a B+ Tree to store terms and their posting lists.

Additional Features

Tokenization

Words can be split into smaller pieces (n-grams) to support typo correction and fuzzy matching.

Ranking

Algorithms such as TF-IDF and BM25 rank results according to relevance rather than simply returning matches.

Vector Indexes (Semantic Search)

Keyword search only matches characters.

For example:

Car $\neq$ Automobile

even though they mean the same thing.

Vector search solves this problem.

Embeddings

A machine learning model converts text into a vector, which is a list of numbers representing semantic meaning.

Example:

"Car" $\rightarrow$ [0.24, -0.11, 0.83, ...]

Similar concepts produce vectors that are close together.

Searching

Instead of looking for exact matches, the database finds vectors nearest to the query vector.

This is known as Approximate Nearest Neighbor (ANN) search.

Album(id, name, year, lyrics)

id	name	year	lyrics
Id1	Enter the Wu-Tang	1993	<text>
Id2	Run the Jewels 2	2015	<text>
Id3	Liquid Swords	1995	<text>
Id4	We Got It from Here	2016	<text>

OpenAI

Hugging Face

Transformer

Embeddings

Id1 → [0.32, 0.78, 0.30, ...]
Id2 → [0.99, 0.19, 0.81, ...]
Id3 → [0.01, 0.18, 0.85, ...]
Id4 → [0.19, 0.82, 0.24, ...]
⋮

Query

Find albums with lyrics about
running from the police
and released after **2005**

[0.02, 0.10, 0.24, ...]

year > 2005

Vector
Index

